

確信度ベース検証順序最適化による 効率的コード生成候補選択

概要: 大規模言語モデル (LLM) によるコード生成では、複数候補を生成し、コンパイルとテストで採用候補を選ぶ運用が一般的である。本研究では、LLM が返す token-level log 確率を用いた候補順位付けを評価し、log 確率がコードの意味的正しさをどこまで予測できるかを調べる。候補全体の平均 log 確率で検証順序を決める Mean Log-Probability Ranking (Mean)、局所的な低確率 token を考慮する Tail Log-Probability Ranking (Tail)、極端な低確信度 token を罰する Extreme-Mean (XMean) を比較する。主評価では、既存ベンチマーク、ParEval 由来の問題、新規問題から 20 問を選抜し、同一プロンプトを独立に呼び出して得た $k = 10$ 個の候補集合上でオフライン評価した。各手法は同一候補集合の検証順序のみを変えるため、成功率は生成順ベースライン Gen と同じ 78.2% である。平均テスト数は Gen の 4.82 回に対して Mean が 4.48 回、Tail が 4.49 回、XMean が 4.42 回であった。20 問を 5 問ずつ 4 ケースに分類すると、LogP-Separable では XMean が平均テスト数を Gen の 3.50 回から 2.52 回へ削減した一方、LogP-Misleading / No-Gain では Gen より悪化した。これらの結果は、log 確率は検証順序の軽量な手がかりにはなるが、意味的誤りの排除には不十分であり、極端な低確信度 token を組み合わせても効果は問題依存であることを示す。

1. はじめに

LLM のコード生成能力は、Codex[1] や AlphaCode[2] 以降、大きく向上している。実用的なコード生成では、単一の出力をそのまま採用するのではなく、複数の候補を生成し、コンパイル、ユニットテスト、性能テストを通して採用候補を選択することが多い。この候補生成とテストに基づく選択は品質を高める一方で、候補数に比例して実行コストが増加する。

特に C 言語性能最適化では、ループ展開、分岐削減、SIMD 命令、キャッシュブロッキング、アルゴリズム変更など、多様な実装方針が存在する。LLM はこれらの候補を複数生成できるが、各候補の品質は一樣ではない。生成順にテストするだけでは、成功候補が後方にある場合に余分なテストが必要となる。一方で、全候補をテストして最良候補を選択する方法は、成功率の観点では堅牢だが、テストコストが高い。

本研究の着眼点は、LLM が各トークンに対して出力する log 確率である。モデルが高い確率で生成したトークン列は、学習済みパターンと整合し、構文的に自然である可能性が高い。ただし、構文的な自然さは意味的正しさを保証しない。そこで、本研究では候補コード全体の平均 log 確率に基づく Mean と、局所的な低確率 token を考慮する Tail を比較し、token log 確率が候補の正しさをどの程度識

別できるかを評価する。

本研究の貢献は以下の 3 点である。

- **log 確率ベース順位付けの限界分析:** 平均 log 確率を用いる Mean と局所 tail 確信度を用いる Tail を比較し、高 log 確率でも意味的に誤る候補が Medium/Hard で多いことを示す。
- **効果が出る問題特性の分析:** 20 問を LogP-Separable, Extreme-Tail Sensitive, Sparse-Success / Limited Signal, LogP-Misleading / No-Gain の 4 ケースに分類し、log 確率順位付けが有効になる条件を分析する。
- **同一候補集合に基づく評価:** 候補生成の乱数差を排除するため、一度生成した候補集合を共有し、検証順序だけがテスト回数に与える影響をオフライン比較する評価手順を構築した。

以下、第 2 章で関連研究、第 3 章で予備知識、第 4 章で提案手法、第 5 章で実験設定、第 6 章で評価結果、第 7 章で考察、第 8 章で結論を述べる。

2. 関連研究

2.1 不確実性を用いた LLM 出力制御

Semantic Entropy[3] は、意味的に同じ出力をクラスタリングし、意味空間でのエントロピーを計算することで LLM の幻覚検出精度を向上させた。Code Semantic Entropy[4] は、この考え方をコード生成に応用し、プログラムの振る

舞いの多様性に基づいて問題レベルの不確実性を測定する。Token-level 不確実性に関しては、TokUR[5] や token-level uncertainty-aware objective[6] が、低ランクランダム重み摂動や aleatoric/epistemic 不確実性の分解により、トークン単位の信頼度評価や訓練目的への応用を検討している。

コード生成への不確実性の応用として、USCD (Uncertainty-Aware Selective Contrastive Decoding) [7] は、token-level 不確実性を用いて出力ノイズを除去し、negative prompt との対比的デコーディングを実施する手法を提案した。また、Sharma ら [8] は、自然言語生成の不確実性推定技術をコード生成に適用し、symbolic execution で意味的等価性をクラスタリングすることで、LLM 出力の正しさを不確実性で評価する手法を提案した。

本研究は、これらの token-level 不確実性計測手法を候補選択の優先順位付けに適用する点に関連する。USCD や symbolic execution に基づく手法がデコーディング時介入や意味的等価性検証に焦点を当てるのに対し、本研究は生成後の候補検証順序の最適化を扱う。

2.2 適応的デコーディング戦略

AdaDec[9] は、token-level 不確実性 (Shannon entropy) を用いたコード生成時の適応的デコーディング手法である。高不確実性を検出したときに pause-then-rerank 機構を実行し、lookahead によりトークン選択を改善する。AdaDec はデコーディング中のリアルタイム介入により単一候補の品質を向上させる手法である。

AdapT[10] や EDT[11] は、トークンごとまたはステップごとに温度を適応的に調整する手法であり、生成品質と多様性のバランスを取る。Semantic uncertainty in advanced decoding methods[12] は、CoT 等の高度なデコーディング手法における semantic uncertainty を調査し、デコーディング戦略が出力の多様性と信頼性に与える影響を分析した。これらの手法がデコーディング時のパラメータ調整に焦点を当てるのに対し、本研究は生成済み候補の検証順序を最適化する。

2.3 候補選択と反復的改善

Self-Consistency[13] は、複数の推論経路をサンプリングし、最も一貫性のある解答を選択する手法で、コード生成でも複数候補選択の有効性を示唆する。Codex[1] や AlphaCode[2] は複数候補を生成し、テスト実行やクラスタリングで絞り込むが、候補の優先順位付けに LLM の確率情報を直接使用していない。CodeT[14] は生成テストを用いて候補を評価し、CodeRL[15] はテスト結果を報酬として利用する。本研究は、LLM の確信度 (token-level 確率の幾何平均) を用いた優先順位付けにより、生成済み候補のテスト回数削減を目指す。

反復的改善に関しては、Self-Refine[16] が LLM 自身によ

る反復的フィードバック・改善を提案したが、コード全体を対象とし、検証順序の最適化は扱わない。UnCert-CoT[17] は不確実性定量化を取り入れ、挑戦的な側面を特定して選択的に CoT 推論を活性化する。AdaDec がデコーディング時のトークンレベル介入であるのに対し、本研究は生成後の候補選択と検証順序最適化を扱い、推論時に得られる token-level 不確実性を利用する。

2.4 確率較正と LLM の信頼性

LLM の確率較正 (calibration) は、出力確率が実際の正解率と一致するかを評価する。Kadavath ら [18] は、言語モデルが自身の知識の境界を一定程度認識できることを示し、Xiong ら [19] は LLM の不確実性表現能力を包括的に評価した。SPUQ[20] は入力摂動により不確実性を推定し、Expected Calibration Error (ECE) を平均 50%削減した。

コード生成タスクでの確率較正の検証はまだ限定的である。本研究は、高確信候補が実際に高品質であるかを、候補検証順序と成功率・テスト回数の観点から評価する。既存研究と比較した本研究の特徴は、(1) 生成後候補集合に対するテスト順序最適化、(2) 生成後検証における軽量な順位付け、(3) 既存の生成パイプラインに後処理として追加できる確信度利用、の3点である。

3. 予備知識

3.1 大規模言語モデルとコード生成

大規模言語モデル (LLM) は、Transformer アーキテクチャを基盤とし、大規模なテキストコーパスとソースコードで事前学習された自己回帰型の生成モデルである。コード生成では、自然言語の問題記述、関数シグネチャ、制約条件、入出力例などを入力として、対応するプログラムをトークン列として生成する。

自己回帰生成では、モデルはそれまでに生成したトークン列 $t_1, \dots, t_{i-1}$ に基づき、次トークン t_i の条件付き確率 $P(t_i | t_1, \dots, t_{i-1})$ を計算する。生成時には、貪欲法、beam search、サンプリングなどにより次トークンが選択される。本研究では複数候補を得るため、温度付きサンプリングを用いる。

温度パラメータ T を用いたサンプリングでは、 logits_i から以下の分布を計算する。

$$P(t_i | t_1, \dots, t_{i-1}) = \frac{\exp(s_i/T)}{\sum_j \exp(s_j/T)} \quad (1)$$

T が大きいほど分布は平坦になり、多様な候補が生成されやすい。一方、 T が小さいほど高確率トークンに集中し、生成は決定的になる。本研究の主評価では、候補の多様性と品質のバランスを取るため、temperature=0.8 を用いる。

3.2 logprobs と確信度計算

vLLM API[21] など、多くの LLM 推論エンジンは、生成各トークンの log 確率 (logprobs) を返す機能を提供する。logprobs は、そのトークンが選択される対数確率 $\log P(t_i|t_1, \dots, t_{i-1})$ であり、モデルが各トークンをどの程度の確信を持って生成したかを表す。

本研究では、候補コード c の確信度を以下で定義する。

$$\text{conf}(c) = \exp \left(\frac{1}{N} \sum_{i=1}^N \log P(t_i) \right) \quad (2)$$

ここで N は生成されたトークン数である。この値は、全トークン確率の幾何平均に相当する。単純な確率積を用いると長い候補ほど不利になるため、平均 log 確率を先に計算し、最後に指数化する。実装では、logprob が取得できない特殊トークンや None 値を除外して計算する。

確信度が候補品質の指標となり得る理由は、以下の 4 点である。

- **学習データとの類似性:** 高確信度は、モデルが学習データ中で頻繁に観測したコードパターンと整合していることを示す。ループ展開や分岐削減のような定型的な最適化は、高確信度で生成されやすい。
- **構文的な一貫性:** 各トークンが高確率で選択されるコードは、局所的な構文や識別子の使い方が一貫している可能性が高い。低確信度トークンが多い場合、構文エラーや関数シグネチャの不一致が生じやすい。
- **実装方針の安定性:** モデルが複数の実装方針の間で迷う場合、トークン確率は分散し、確信度が低下する。アルゴリズム変更やデータ構造実装では、境界条件、再帰終了条件、メモリ管理などでこの傾向が現れる。
- **経験的相関:** Kadavath ら [18] は、LLM の確率出力と正解率に相関があることを示した。本研究の評価においても、成功候補は高確信度側に多く、失敗候補は低確信度側に多い傾向が確認された (詳細は第 6 章)。

3.3 vLLM API

vLLM[21] は、PagedAttention を用いた高速な LLM 推論エンジンであり、OpenAI 互換 API として利用できる。本研究では、Qwen3.5-9B-AWQ-4bit を vLLM 経由で利用した。vLLM API では、logprobs パラメータを指定することで、生成トークンとともに上位候補トークンの log 確率を取得できる。

本手法の実装では、各候補について生成テキストと token-level logprobs を同時に取得する。その後、コードブロックを抽出し、候補単位の確信度を計算する。この処理は生成後に行うため、デコーディングアルゴリズム自体を変更する必要がない。したがって、log 確率に基づく検証順序最適化は既存の vLLM ベースのコード生成パイプラインに後処理として追加できる。

3.4 性能最適化タスク

本研究の評価対象は、C 言語の性能最適化タスクである。各問題では、指定された関数シグネチャに従って関数を実装し、コンパイルとテストに通過することを成功条件とする。問題には、配列和や疎行列ベクトル積のループ最適化、分岐削減、SIMD 風のベクトル演算、混合精度計算、ソートやグラフ計算などが含まれる。ループ展開は複数回の反復をまとめてループ制御オーバーヘッドを削減するが、剰余要素の処理を誤ると正しさを損なう。分岐削減は条件分岐を算術演算等に置き換えるが、境界条件の実装を誤ると結果が変わる。アルゴリズム変更は計算量の異なる実装へ置き換えるが、関数全体の構造、補助関数、再帰、境界条件を正しく実装する必要がある。LLM にとって難度が高い。また、ステンシル計算や混合精度正規化のように、構文的には自然でも数値誤差や境界条件で失敗しやすい問題も含まれる。

4. 提案手法

4.1 log 確率に基づく検証順序最適化

本稿では、LLM が出力する token-level log 確率から候補コードの確信度を計算し、その確信度に基づいて検証順序を制御する。扱う選択規則は、平均 log 確率を用いる Mean Log-Probability Ranking (Mean)、局所的な低確率 token を考慮する Tail Log-Probability Ranking (Tail)、極端な低確信度 token を罰する Extreme-Mean (XMean) である。

- **Mean:** 固定候補数 k 個の候補を生成し、平均 log 確率に基づく確信度順に検証する。
- **Tail:** 固定候補数 k 個の候補を生成し、token log 確率の下位四分位に基づいて検証する。
- **XMean:** 固定候補数 k 個の候補を生成し、平均 log 確率から極端な低確信度 token の割合を罰則として差し引いたスコアで検証する。

Mean, Tail, XMean はいずれも同じ固定候補集合に対する検証順序制御であり、候補集合そのものは Gen と共有する。そのため、候補集合中に成功候補が存在するかどうかは全手法で同じであり、成功率は検証順序に依存しない。本稿の主目的は、成功率ではなく、成功候補に到達するまでのテスト回数を削減できるかを評価することである。

4.2 Mean, Tail, XMean

Mean は、固定候補数 k 個の候補を生成し、平均 log 確率に基づく確信度順に検証することで、成功候補に到達するまでの平均テスト数を削減する手法である。同一候補集合を用いるため成功率はベースライン (全候補を検証) と同等だが、検証順序を最適化することで無駄なテストを削減できる。

Algorithm 1 に Mean を示す。まず LLM から k 個の候

Algorithm 1 Mean Log-Probability Ranking

Require: 問題記述 P , 候補数 k

Ensure: 成功したコード候補または失敗

```

1: 候補集合  $C = \{c_1, \dots, c_k\}$  を LLM で生成し, logprobs を取得
2: for 各候補  $c_j \in C$  do
3:    $\text{conf}(c_j) = \exp(\frac{1}{N_j} \sum_i \log P(t_i))$  を計算
4: end for
5:  $C$  を  $\text{conf}(c_j)$  の降順に整理
6: for 各候補  $c_j \in C$  do
7:    $c_j$  をコンパイルし, テストを実行
8:   if  $c_j$  が成功 then
9:     return  $c_j$ 
10:  end if
11: end for
12: return 失敗
  
```

補を生成し, 各候補の logprobs から平均 log 確率に基づく確信度を計算する. 次に候補を確信度の降順に並べ替え, 順にコンパイル・テストする. 成功候補が見つければ, その時点で終了し, 残りの候補の検証を省略する. 全候補が失敗した場合は失敗として扱う.

この手法は, 候補プール内の成功候補が高確信度側に偏っている場合に有効である. 例えば, $k = 10$ の候補のうち確信度 1 位が成功候補であれば, テスト数は 1 回で済む. 一方, 成功候補が確信度 10 位であれば, テスト数は 10 回となる. したがって, 平均テスト数は成功候補の確信度順位に依存する.

平均 log 確率は候補全体の自然さを表す一方で, 局所的に極端に低確率な token の存在を十分に反映しない場合がある. そこで本稿では, 固定候補集合の別の順位付け規則として Tail を定義する. Tail では, 候補 c の token log 確率列 $\{\ell_i\}_{i=1}^N$ に対し, 下位四分位

$$\text{tail}(c) = Q_{0.25}(\{\ell_i\}_{i=1}^N) \quad (3)$$

を局所 tail 確信度として用い, $\text{tail}(c)$ の降順に候補を検証する. これは, 平均的には自然でも一部に低確率 token を含む候補を後回しにすることを意図した規則である.

さらに, 全 token 分布の分析から, 失敗候補では下位四分位よりもさらに右端の極端な低確信度 token が有効な手がかりになる場合があることが分かった. そこで XMean では, $\text{loss} - \ell_i$ が 6 以上となる token の割合

$$r_6(c) = \frac{1}{N} |\{i \mid -\ell_i \geq 6\}| \quad (4)$$

を用い,

$$\text{xmean}(c) = \frac{1}{N} \sum_{i=1}^N \ell_i - 100r_6(c) \quad (5)$$

の降順に候補を検証する. これは, 平均的には高確信度でも局所的に極端に怪しい token を含む候補を後回しにすることを意図した規則である.

5. 実験設定

5.1 環境

実験では, Qwen3.5-9B-AWQ-4bit を vLLM[21] 経由で利用し, OpenAI 互換 API から logprobs を取得した. 生成パラメータは temperature=0.8, 候補数は全評価で $k = 10$ に統一した. 推論には chat completion API を用い, thinking 出力は chat template の設定で無効化した. max_tokens は 4096 とした. 生成された C コードは gcc 14.3.1 で -O3 -lm を付けてコンパイルし, 各問題に付属するテストプログラムを実行した. 実験環境は Linux 6.12, Python 3.12.12, Intel Xeon Platinum 8368 CPU である. 主評価では 20 問それぞれについて 50 試行を行い, 合計 1000 試行を評価した. コード生成時の乱数 seed は vLLM API 側で固定していないため, 完全な再生成再現性は保証しない. 一方, 主評価では全候補のコード, 確信度, コンパイル・テスト結果, エラー種別を JSON に保存し, 解析の再現性を確保した.

5.2 比較手法

ここで, 主評価における候補の生成方法を明示する. 各問題には, 5.4 節で述べる通常の最適化指示プロンプト P がある. これは問題の目的, 関数シグネチャ, 実装上の制約, “Return only C code.”, “Required #include directives and helper definitions are allowed.”, “Do not include markdown fences or explanatory text.” という最小限の出力制約を含む基本プロンプトである. 主評価では, プロンプト内で複数解の列挙を要求するのではなく, 各問題・各試行について同じプロンプト P を $k = 10$ 回独立に API へ送信し, 1 回の呼び出しから 1 個の候補を得る. これにより候補集合 $C = \{c_0, \dots, c_{k-1}\}$ を構成する.

本稿の比較手法はすべて, この同一候補プールに対する**選択規則**の違いとして定義する. 実運用時の各手法は成功候補が見つかった時点で停止するが, 実験では候補集合を固定して比較するため, まず全候補をコンパイル・テストして候補単位の成否と log 確率統計を保存し, その後に各規則をオフライン評価する.

- **Gen:** 同一プロンプトから得た共有候補プールを API の返却順 $c_0, \dots, c_{k-1}$ にテストし, 最初に成功した候補を採用する生成順早期停止である.
- **Mean:** 共有候補プール全体を使い, 平均 log 確率に基づく確信度の降順に候補をテストする Mean Log-Probability Ranking である.
- **Tail:** 共有候補プール全体を使い, token log 確率の下位四分位 $\text{tail}(c)$ の降順に候補をテストする Tail Log-Probability Ranking である.
- **XMean:** Mean に極端な低確信度 token の割合を罰

則として加える Extreme-Mean である。候補 c の平均 $\log$ 確率を $\bar{\ell}(c)$, $-\ell_i \geq 6$ となる token 割合を $r_6(c)$ とし, $\bar{\ell}(c) - 100r_6(c)$ の降順に候補をテストする。

この設計により, 候補集合の違いによる成功率の揺らぎを排除し, 検証順序だけがテスト数と採用候補に与える影響を評価できる。Gen, Mean, Tail, XMean はいずれも同じ候補集合を成功候補が見つかるまで, または全候補が失敗するまで検証する。したがって, 各試行の成功条件は「候補集合中に少なくとも 1 個の成功候補が存在すること」であり, 成功率は手法間で同一である。

5.3 ベンチマーク

本研究では, 既存ベンチマーク, ParEval 由来の問題, 新規に作成した数値・検証問題から 20 問を選抜した。本稿では, コードの題材だけでなく, $\log$ 確率に基づく順位付けがどのように効いたかを分析するため, 20 問を 5 問ずつ 4 ケースに分類する。これは事前難易度ではなく, 候補集合上で観察された確信度信号の振る舞いに基づく分析用分類である。

- **LogP-Separable:** Mean または Tail だけで Gen より明確に少ないテスト数を達成した問題群である。成功候補と失敗候補の $\log$ 確率統計が比較的分離する。
- **Extreme-Tail Sensitive:** 下位四分位だけでは十分でないが, 極端に低確信度な token を罰する XMean が改善しやすい問題群である。
- **Sparse-Success / Limited Signal:** 候補集合内の成功候補が少ない, または成功候補があっても $\log$ 確率信号が弱く, 改善余地が限定される問題群である。
- **LogP-Misleading / No-Gain:** $\log$ 確率に基づく順位付けがほぼ効かない, または Gen より悪化する問題群である。意味的誤りが自然な token 列として生成される場合を含む。

この構成により, 単に成功率の高低で分けるのではなく, Mean, Tail, XMean がどのような候補集合で有効になるかを分析できる。表 1 に各ベンチマークの詳細を示す。

5.4 プロンプト設計

本研究で LLM に生成させるコードは, 完全なプログラムではなく, 指定された C 関数または関連する関数群の実装である。各問題にはテスト用の main 関数を含むテストプログラムが付属しており, LLM の出力コードをテストプログラムとリンクしてコンパイル・実行する。したがって, 生成候補は関数シグネチャを満たし, テストプログラムが期待する副作用や戻り値を実装する必要がある。

生成プロンプトは, 各問題の目的, 関数シグネチャ, 実装上の指示, 出力制約からなる。典型的には, “Implement an optimized ... in C” で最適化対象を指定し, “Function signature:” または “Function signatures:” で必要な関数を

列挙する。続いて, ループ展開, 分岐削減, キャッシュ局所性, 配列ベースデータ構造, 数値安定化など, 期待する実装方針を自然言語で与える。最後に, C コードのみを出力し, 必要な `#include` と補助定義は許可し, Markdown フェンスや説明文を含めないという最小限の制約を付ける。

例えば, Strided Transpose では, 入力要素 `in[i*in_stride+j]` を出力要素 `out[j*out_stride+i]` へ書き込み, padding 領域を変更しないよう指示する。Radix では, キーと値の対応を保持した安定ソートを実装するよう指示する。Sort Nonzero では, ゼロ値の位置を保持しながら非ゼロ要素のみを整列する制約を明示する。RMSNorm では, float 入出力に対して double 蓄積を用い, テストで要求される誤差閾値を満たすよう指示する。UTF-8 検証では, 過長符号化や surrogate を拒否する境界条件を明示し, Cholesky 分解では非正定値入力を検出して失敗を返すよう指示する。Convex Hull では重複点や共線点の扱いを明示する。このように, プロンプトは単なる関数名だけでなく, 期待する最適化方針と正しさに必要な条件を含む。主評価の共有候補プールでは, 同一の問題プロンプトを独立に 10 回呼び出して候補を生成し, 候補間の改善指示は用いない。したがって, 主評価の候補集合は段階的な改善系列ではなく, 同一プロンプト条件下で得た 10 候補である。

5.5 評価指標

成功率は, コンパイルに成功し, テストプログラムが PASS を出力した候補を採用できた試行の割合である。本稿の比較手法は同一候補集合の順序だけを変えるため, 成功率は手法別の性能差ではなく, 候補集合自体の品質を表す補助指標である。平均テスト数は, 各選択規則が成功または全失敗を確定するまでに必要とする候補テスト数を, 問題数と試行数で割った値である。全候補検証では常に k となり, 早期停止方式では成功候補の順位に応じて小さくなる。

Mean および Tail は生成済み候補の検証順序を決める手法であり, 候補集合内に成功候補が存在しない試行を成功させることはできない。そこで, 候補生成品質を表す指標として, 全候補を検証したときに少なくとも 1 個の成功候補が存在した試行数も確認する。

6. 評価

6.1 主要評価

各手法を同一候補プール上で評価する。図中では, 生成順ベースラインを Gen, 平均 $\log$ 確率順位を Mean, 局所 tail 順位を Tail, 極端な低確信度 token を罰する順位を XMean と略記する。4 ケースの評価結果を図 1-4 に示す。上段の確信度分布では破線が $\text{confidence}=0.75$ を示し, タイトル下の括弧内にその閾値以上での成功率を示す。また,

表 1 生成対象となる C コードの内容 (20 課題)

ベンチマーク	集合	カテゴリ	生成対象コード
Sort Nonzero	LogP-Separable	Sorting	ゼロ位置を固定したまま、非ゼロ要素のみを昇順に並べ替える。
UTF-8 Validate	LogP-Separable	Validation	UTF-8 列について過長符号化, surrogate, 範囲外 code point を厳密に拒否する。
Largest Component	LogP-Separable	Graph	グリッドまたはグラフ上の連結成分を探索し、最大成分サイズを返す。
Crop2D	LogP-Separable	Stride	入力 stride と出力 padding を持つ 2 次元 crop を実装し、padding 領域を変更しない。
SpMV	LogP-Separable	Sparse	CSR 形式の疎行列ベクトル積, AXPY, 内積を一回の走査で融合する。
Conv2D	Extreme-Tail	Convolution	NHWC 配置の multi-channel 3x3 valid convolution を実装する。
Transpose	Extreme-Tail	Cache	行列転置をブロック単位に分割してキャッシュ局所性を高める。
Radix	Extreme-Tail	Sorting	キーと値の対応を保持する安定 radix sort を実装する。
RMSNorm	Extreme-Tail	Reduction	RMSNorm の二乗和計算を double で蓄積して丸め誤差を抑える。
HeapSort	Extreme-Tail	Heap	ヒープ構造の構築・維持とソート処理を正しく実装する。
Banded Edit	Sparse/Limited	Dynamic Prog.	帯幅制約付き編集距離を計算し、帯外遷移を扱う。
Convex Hull	Sparse/Limited	Geometry	重複点や共線点を含む点集合に対して凸包周長を計算する。
Top-k NaN	Sparse/Limited	Selection	NaN を無視して浮動小数点配列の上位 k 要素を選択する。
Stencil3D	Sparse/Limited	Stencil	3 次元 7 点ステンシルを境界条件と精度制約を満たして実装する。
Stencil2D	Sparse/Limited	Stencil	halo 領域を含む 2 次元ステンシル計算を実装する。
Floyd	Misleading/No-Gain	Graph	全点对最短路を Floyd-Warshall アルゴリズムで in-place 更新する。
Strided Transpose	Misleading/No-Gain	Stride	入出力 stride を持つ矩形行列転置を実装し、出力 padding を保持する。
Triangular Solve	Misleading/No-Gain	Solver	複数次右辺の下三角ソルバを stride 付き行列上で実装する。
Quadratic Roots	Misleading/No-Gain	Numerics	桁落ちを避ける二次方程式の安定な解法を実装する。
Cholesky SPD	Misleading/No-Gain	Linear Algebra	対称正定値行列の Cholesky 分解を行い、非正定値入力を検出する。

高確信度領域にある失敗候補を赤い rug マークで示す。下段は trial 単位のテスト数の箱ひげ図である。箱は四分位範囲、中央線は中央値、ヒゲは 1.5 IQR 内の範囲、色付きひし形は平均値を示す。

図 5 は、Mean と Tail が実際に参照する 2 つの log 確率統計量を候補単位で可視化したものである。成功候補と失敗候補が右上方向で分離している問題では、log 確率に基づく順位付けが有効に働く。一方、成功候補と失敗候補が大きく重なる問題では、高確信度でも意味的に誤る候補が多く、log 確率だけでは検証前に失敗を十分に排除できない。Tail が Mean より高い AUC を示す問題では、候補全体の平均的な自然さよりも、局所的な低確率 token の少なさが成功候補の順位付けに有効であることを示している。図 6 は、同じ現象を token 単位で見たものである。失敗候補の曲線が成功候補より上側にある領域では、失敗候補が低確信度 token を多く含んでいることを意味する。今回の 20 問分析では、 $\text{loss} \geq 4$ となる低確信度 token 割合について、失敗候補と成功候補の差と Tail の平均テスト数削減量の相関は約 0.55 であった。これは、低確信度 token の裾の差が Tail の効果を部分的に説明する一方、それだけでは十分でないことを示す。実際、Sort Nonzero, UTF-8 Validate, Largest Component のように失敗候補の裾が相対的に重い問題では Tail が有効であったが、Floyd や Strided Transpose のように曲線が重なる、または成功候補側も同程度に低確信度 token を含む問題では Tail が悪化した。

図 1-4 より、log 確率に基づく順位付けの効果はケースによって大きく異なる。LogP-Separable では Gen の平均 3.50 回に対し、Mean は 2.51 回、Tail は 2.54 回、XMean は 2.52 回まで削減した。Extreme-Tail Sensitive では XMean が最良で、Gen の 3.80 回を 3.39 回へ削減した。一方、Sparse-Success / Limited Signal では Tail が 7.34 回を 6.92 回へ削減したものの、候補集合内の成功候補が少ないためテスト数は高く残った。LogP-Misleading / No-Gain では、Mean と Tail は Gen より悪化し、XMean でも Gen を上回れなかった。

6.2 全体集計

20 問 (全 1000 試行) で集計すると、候補プール内に少なくとも 1 個の成功候補が存在した試行は 78.2% であった。Gen, Mean, Tail, XMean はいずれも同じ候補集合を全候補まで検証可能なため、成功率はこの候補プール成功率と一致する。Mean は平均テスト数を Gen の 4.82 回から 4.48 回へ削減し、Tail は 4.49 回、XMean は 4.42 回であった。

6.3 Temperature と候補数の感度分析

Tail の効果が生成設定に依存するかを確認するため、Tail の効果が比較的大きな `transpose_strided.f64` と `utf8_validate_strict` の 2 問を用いて、temperature と候補数 k を変えた感度分析を行った。各条件では 20 試行

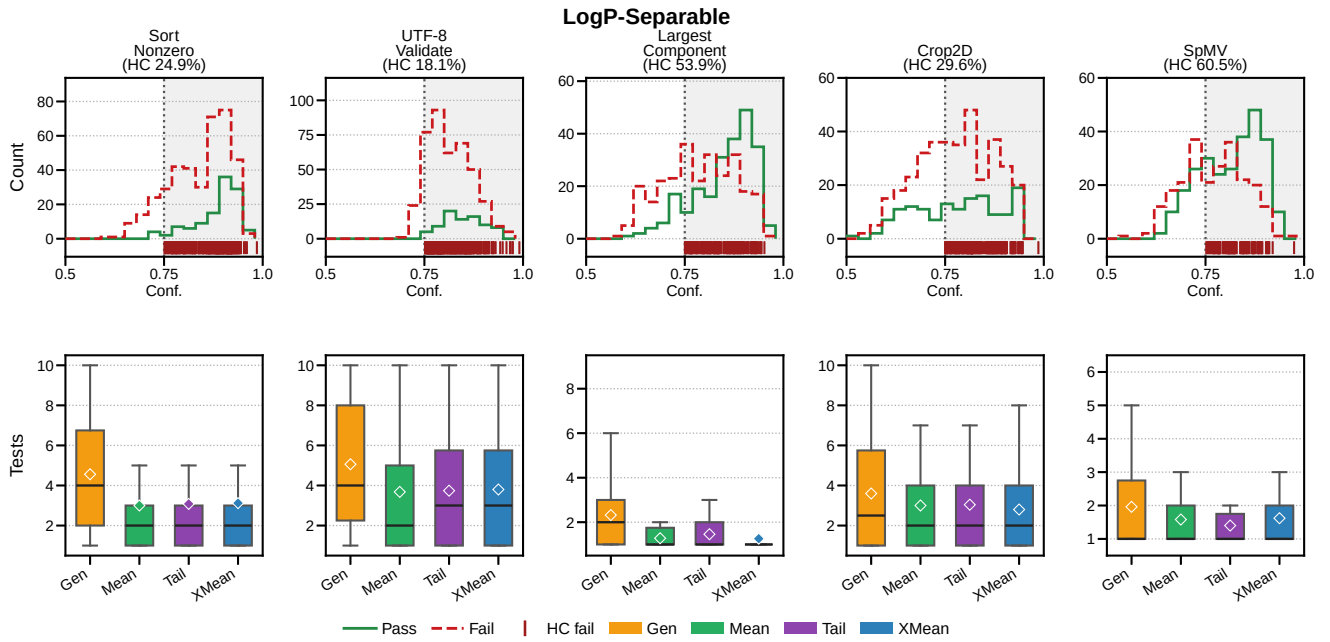


図 1 LogP-Separable ケース (Sort Nonzero, UTF-8 Validate, Largest Component, Crop2D, SpMV) の評価結果. 成功候補と失敗候補の log 確率統計が比較的分離し, Mean, Tail, XMean が Gen より大きくテスト数を削減した.

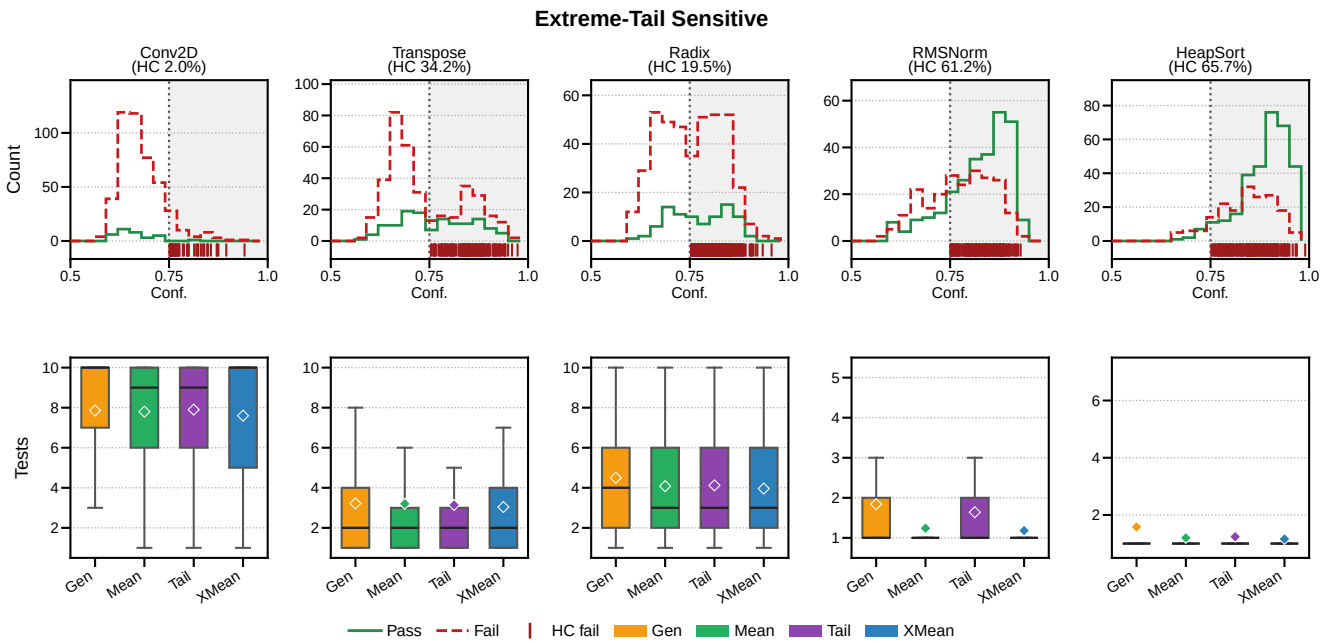


図 2 Extreme-Tail Sensitive ケース (Conv2D, Transpose, Radix, RMSNorm, HeapSort) の評価結果. 下位四分位だけでは十分でないが, 極端に低確信度な token を罰する XMean が改善しやすい問題群である.

ずつ, 合計 40 試行を評価した. 表 4 に結果を示す.

全体として, k を大きくすると Tail が検証順序を改善する余地が大きくなった. Gen では最初の 5 候補内に成功候補がある試行のテスト数は $k = 5$ と $k = 10$ で変わらないため, $k = 10$ での改善は Gen が大きく悪化したためではない. Tail は候補集合全体を順位付けするため, k が

大きいほど選択可能な成功候補が増え, 成功候補を上位に配置できる機会が増える. 特に $k = 10$ では Tail gain が 0.52–0.55 となり, $k = 3$ や $k = 5$ より大きい改善が見られた. 一方, temperature=1.0 では $k = 3, 5$ で Tail gain が 0.07–0.10 に低下し, 高 temperature で候補品質のばらつきが大きくなると, log 確率だけによる順位付けが不安定

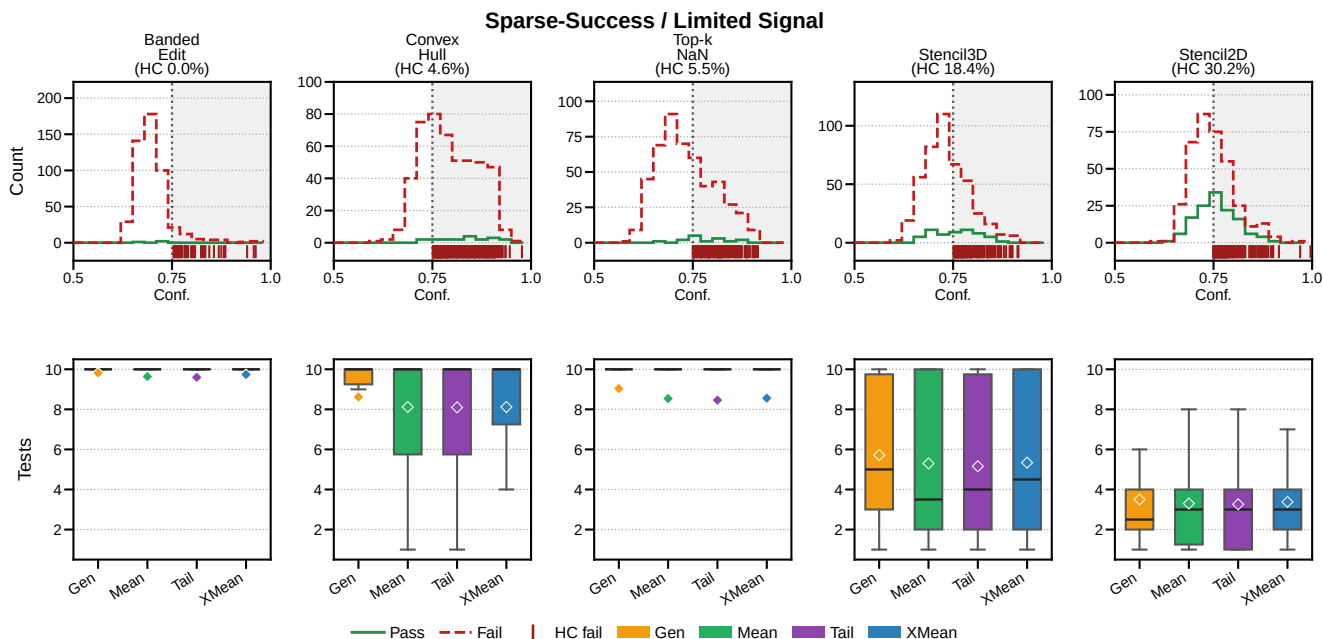


図 3 Sparse-Success / Limited Signal ケース (Banded Edit, Convex Hull, Top-k NaN, Stencil3D, Stencil2D) の評価結果. 候補集合内の成功候補が少ない, または log 確率信号が弱く, 改善後もテスト数が高めに残る.

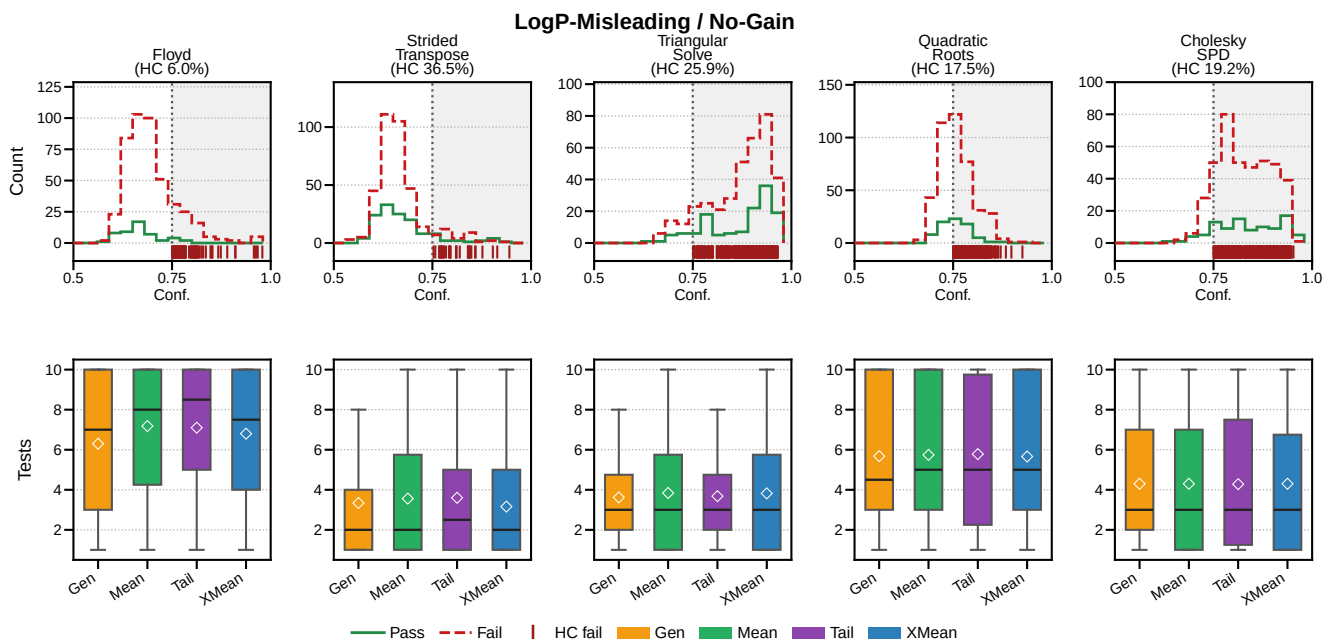


図 4 LogP-Misleading / No-Gain ケース (Floyd, Strided Transpose, Triangular Solve, Quadratic Roots, Cholesky SPD) の評価結果. log 確率に基づく順位付けがほぼ効かない, または Gen より悪化する問題群である.

になることが示唆される. 本実験では temperature=0.8 が最も安定して Tail の改善を示した. ただし, 本感度分析は 2 問, 各条件 40 試行の予備実験であり, 主評価とは区別して解釈する必要がある.

7. 考察

各手法は同じ token-level log 確率を使うが, 利用する統計量が異なる. Mean は平均 log 確率に基づく検証順序最適化の基本形であり, Tail は局所的な低確率 token を考慮する順位付け規則である. 結果として, Tail は Mean より

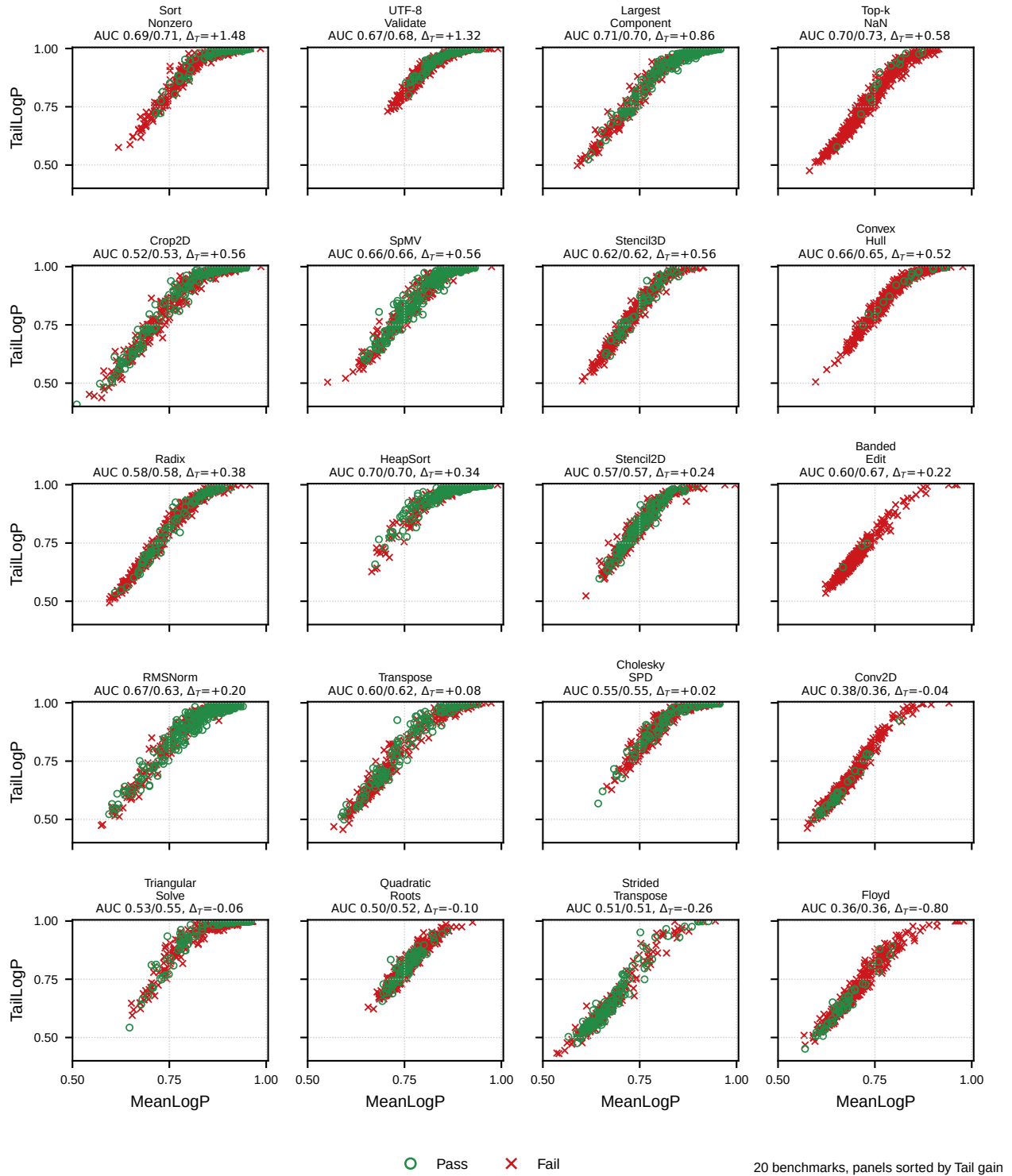


図 5 各候補の MeanLogP と TailLogP の分布. 9B モデル, $k = 10$ で評価した 20 ベンチマークについて, 各点は 1 候補を表し, 横軸は平均 token log 確率に基づく確信度, 縦軸は token log 確率の下位四分位に基づく tail 確信度である. 緑はテスト成功候補, 赤は失敗候補を示す. 各パネルの AUC は, 成功候補を失敗候補より高く順位付けできる確率を Mean/Tail の順に示し, Δ_T は Gen に対する Tail の平均テスト数削減量を表す. 各パネルは Δ_T の降順に並べた.

平均テスト数をわずかに削減したが, log 確率だけで高確信度の意味的誤りを十分に排除するには至らなかった.

本評価では成功率が Gen, Mean, Tail で一致するが, これは手法の効果ではなく評価設計から従う性質である. 3

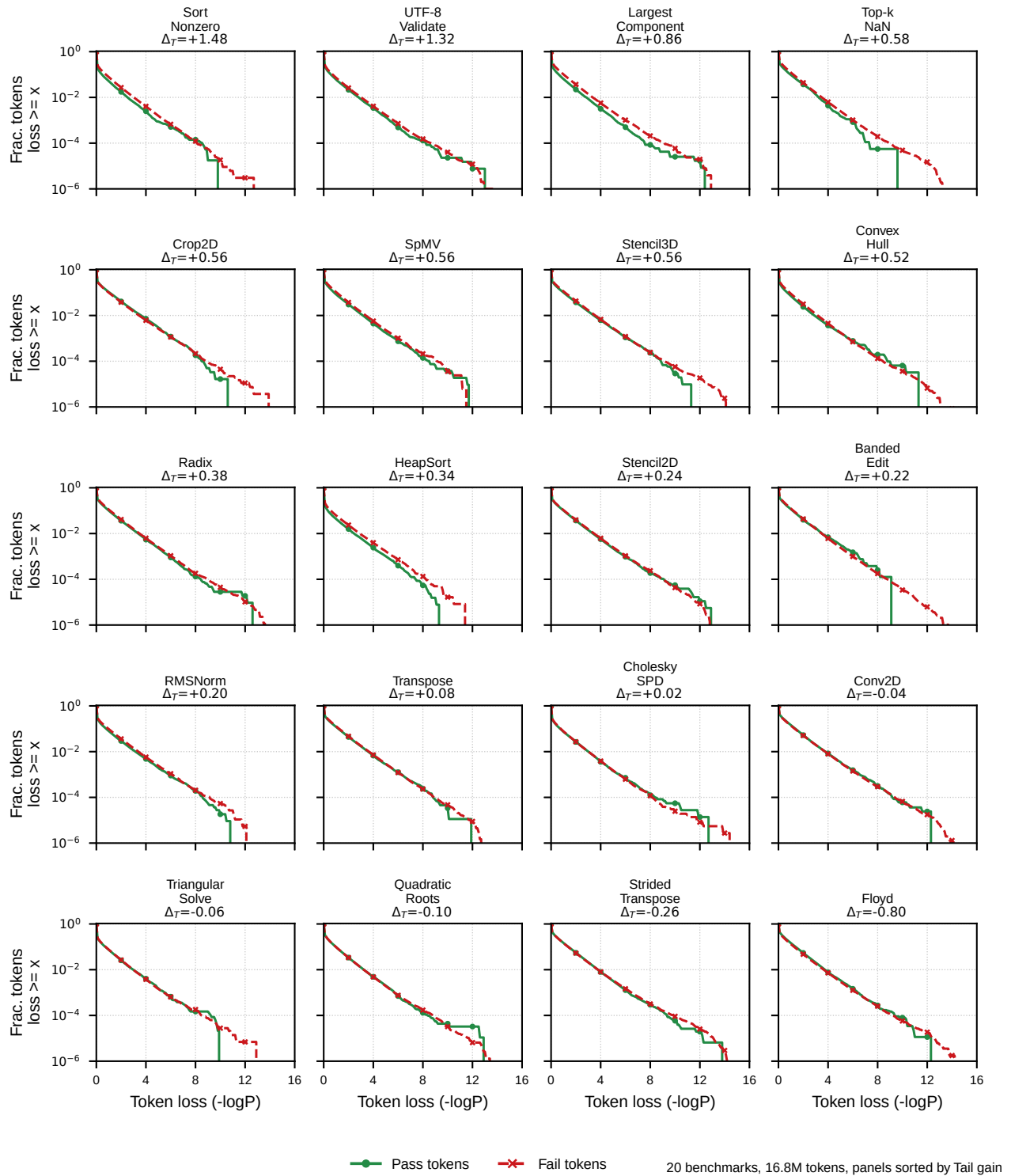


図 6 全 token の低確信度側分布. 9B モデル, $k = 10$ で評価した 20 ベンチマークについて, 成功候補に含まれる token と失敗候補に含まれる token を集約し, token loss $-\log P(t)$ の生存分布を示す. 縦軸は loss が横軸値以上となる token の割合であり, 右側の裾が長いほど低確信度 token を多く含む. 各パネルは Tail による平均テスト数削減量 Δ_T の降順に並べた. 全 token を点で描くと過密になるため, 約 1680 万 token を分布として集約した.

手法はいずれも同一候補集合を成功候補が見つかるまで検証するため, 成功条件は候補集合中に成功候補が存在する

ことと等価である. したがって, 本稿で比較すべき主指標は成功率ではなく, 成功候補に到達するまでの平均テスト

表 2 主評価 20 問の手法別集計

手法	平均テスト数
Gen	4.82
Mean	4.48
Tail	4.49
XMean	4.42

数である。

本研究で用いた確信度はトークン確率の幾何平均，下位四分位，または極端な低確信度 token 割合であり，軽量で実装しやすい一方，コードの意味的正しいさを直接評価するものではない。LogP-Separable では成功候補と失敗候補の log 確率統計が比較的分離し，検証順序最適化が有効であった。Extreme-Tail Sensitive では，平均や下位四分位だけでは捉えにくい局所的な極端低確信度 token を罰する XMean が有効であった。一方，LogP-Misleading / No-Gain では，更新順序，数値精度，境界条件などの意味的誤りが自然な token 列として生成され得るため，log 確率だけでの識別は限定的である。今後は，構文解析，型検査，静的解析，生成テストなどを組み合わせた確信度拡張が有望である。

Mean と Tail は logprobs を取得し候補をソートするだけであるため，既存の vLLM ベースのコード生成パイプラインに容易に追加できる。一方，本稿の評価は固定候補集合上のオフライン評価であり，生成候補数や壁時計時間の削減は対象外である。実運用での総実行時間を議論するには，生成時間，コンパイル時間，テスト時間を含めたオンライン評価が別途必要である。

8. 結論

本研究では，LLM の token-level log 確率を用いた候補順位付けを評価した。具体的には，固定数の候補を平均 log 確率順に検証する Mean と，局所 tail 確信度で順位付ける Tail を比較した。

評価の結果，Mean，Tail，XMean は，候補集合内に成功候補が含まれる場合に，平均テスト数を削減できるケースがあることを確認した。20 問全体では，Gen の平均 4.82 回に対して Mean は 4.48 回，Tail は 4.49 回，XMean は 4.42 回であった。特に LogP-Separable では平均テスト数を Gen の 3.50 回から XMean の 2.52 回へ削減し，Extreme-Tail Sensitive では Gen の 3.80 回から XMean の 3.39 回へ削減した。一方，LogP-Misleading / No-Gain では高 log 確率でも意味的に誤る候補が残り，log 確率だけで semantic correctness を十分に判別できない。

以上より，token log 確率は複数候補を生成してテストで選別するコード生成において有用な軽量信号であるが，意味的正しいさの判定には限界がある。本稿の 20 問分類は，log 確率信号が有効なケースと誤誘導するケースの両方を

含む評価であり，より強い一般化には追加問題，同一試行数での再評価，オンライン評価が必要である。今後は，log 確率に静的解析，生成テスト，仕様一致チェックを組み合わせ，高確信度だが意味的に誤る候補をテスト前に検出する方法へ拡張する必要がある。

参考文献

- [1] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G. et al.: Evaluating Large Language Models Trained on Code, Technical Report arXiv:2107.03374, OpenAI (2021).
- [2] Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., Hubert, T., Choy, P., de Masson d’Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Goyal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Sutherland Robson, E., Kohli, P., de Freitas, N., Kavukcuoglu, K. and Vinyals, O.: Competition-level code generation with AlphaCode, *Science*, Vol. 378, No. 6624, pp. 1092–1097 (online), DOI: 10.1126/science.abq1158 (2022).
- [3] Farquhar, S., Kossen, J., Kuhn, L. and Gal, Y.: Detecting hallucinations in large language models using semantic entropy, *Nature*, Vol. 630, No. 8017, pp. 625–630 (online), DOI: 10.1038/s41586-024-07421-0 (2024).
- [4] Zhang, H., Cheng, W. and Hu, W.: Self-Improving Code Generation via Semantic Entropy and Behavioral Consensus, Technical Report arXiv:2603.29292, arXiv (2026).
- [5] Zhang, T., Shi, H., Wang, Y., Wang, H., He, X., Li, Z., Chen, H., Han, L., Xu, K., Zhang, H., Metaxas, D. and Wang, H.: TokUR: Token-Level Uncertainty Estimation for Large Language Model Reasoning, *Proc. 14th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=VHQc7wzmYv> (2026).
- [6] Liu, T., Benjamin, A. S. and Zador, A. M.: Token-Level Uncertainty-Aware Objective for Language Model Post-Training, Technical Report arXiv:2503.16511, arXiv (2025).
- [7] Wang, S., Ding, L., Shen, L., Luo, Y., He, Z., Yu, W. and Tao, D.: USCD: Improving Code Generation of LLMs by Uncertainty-Aware Selective Contrastive Decoding, Technical Report arXiv:2409.05923, arXiv (2024).
- [8] Sharma, A. and David, C.: Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation, Technical Report arXiv:2502.11620, arXiv (2025).
- [9] He, K., Liu, M., Wang, C., Li, Z., Wang, Y., Peng, X. and Zheng, Z.: Towards Better Code Generation: Adaptive Decoding with Uncertainty Guidance, Technical Report arXiv:2506.08980, arXiv (2025).
- [10] Zhu, Y., Li, J., Li, G., Zhao, Y., Li, J., Jin, Z. and Mei, H.: Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models, *Proc. AAAI Conference on Artificial Intelligence (AAAI)*, (online), available from <https://arxiv.org/abs/2309.02772> (2024).
- [11] Zhang, S., Bao, Y. and Huang, S.: EDT: Improving Large Language Models’ Generation by Entropy-based Dynamic Temperature Sampling, Technical Report arXiv:2403.14541, arXiv (2024).

表 3 主評価における問題群別結果

分類	試行数	Pool success	First	Gen	Mean	Tail	XMean
LogP-Separable	250	94.8%	33.2%	3.50	2.51	2.54	2.52
Extreme-Tail Sensitive	250	87.6%	38.8%	3.80	3.50	3.61	3.39
Sparse-Success / Limited Signal	250	46.0%	5.2%	7.34	6.98	6.92	7.03
LogP-Misleading / No-Gain	250	84.4%	16.8%	4.65	4.92	4.89	4.75

表 4 Temperature と候補数 k に対する感度分析 (2 問, 各条件 40 試行)

temperature	k	Pool success	Gen tests	Mean tests	Tail tests	Tail gain
0.4	3	95.0%	1.45	1.23	1.18	0.27
0.4	5	97.5%	1.60	1.30	1.40	0.20
0.4	10	100.0%	1.62	1.07	1.10	0.52
0.8	3	97.5%	1.43	1.23	1.15	0.28
0.8	5	97.5%	1.70	1.32	1.30	0.40
0.8	10	100.0%	1.80	1.15	1.25	0.55
1.0	3	95.0%	1.52	1.38	1.45	0.07
1.0	5	97.5%	1.65	1.57	1.55	0.10
1.0	10	100.0%	1.82	1.32	1.30	0.52

- [12] Foodei, D., Fan, S. and Jaggi, M.: Semantic uncertainty in advanced decoding methods for LLM generation, Technical Report arXiv:2506.17296, arXiv (2025).
- [13] Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A. and Zhou, D.: Self-Consistency Improves Chain of Thought Reasoning in Language Models, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=1PL1NIMMrw> (2023).
- [14] Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G. and Chen, W.: CodeT: Code Generation with Generated Tests, *Proc. 11th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=ktrw68Cmu9c> (2023).
- [15] Le, H., Wang, Y., Gotmare, A. D., Savarese, S. and Hoi, S. C. H.: CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning, *Advances in Neural Information Processing Systems 35 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2207.01780> (2022).
- [16] Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A. and Clark, P.: Self-Refine: Iterative Refinement with Self-Feedback, *Advances in Neural Information Processing Systems 36 (NeurIPS)*, (online), available from <https://arxiv.org/abs/2303.17651> (2023).
- [17] Zhu, Y., Li, G., Jiang, X., Li, J., Mei, H., Jin, Z. and Dong, Y.: Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs, Technical Report arXiv:2503.15341, arXiv (2025).
- [18] Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E. et al.: Language Models (Mostly) Know What They Know, Technical Report arXiv:2207.05221, arXiv (2022).
- [19] Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J. and Hooi, B.: Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs, *Proc. 12th International Conference on Learning Representations (ICLR)*, (online), available from <https://openreview.net/forum?id=gjeQKFxFpZ> (2024).
- [20] Gao, X., Zhang, J., Mouatadid, L. and Das, K.: SPUQ: Perturbation-Based Uncertainty Quantification for Large Language Models, *Proc. 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2336–2346 (online), DOI: 10.18653/v1/2024.eacl-long.143 (2024).
- [21] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H. and Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention, *Proc. 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (online), available from <https://arxiv.org/abs/2309.06180> (2023).